

作业 HW2 实验报告

姓名：庄子硕 学号：2350998 日期：2024 年 10 月 14 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

本次作业考察栈和队列的构造和使用。

2. 实验内容

2.1 列车进站

2.1.1 问题描述

每一时刻，列车可以从入口进车站或直接从入口进入出口，再或者从车站进入出口。即每一时刻可以有一辆车沿着箭头 a 或 b 或 c 的方向行驶。现在有一些车在入口处等待，给出该序列，然后给你多组出站序列，请你判断是否能够通过上述的方式从出口出来。

2.1.2 基本要求

输入要求：

第 1 行，一个串，进站序列。

后面多行，每行一个串，表示出栈序列

当输入=EOF 时结束

输出要求：

多行，若给定的出栈序列可以得到，输出 yes,否则输出 no

2.1.3 数据结构设计

定义了顺序栈类型，和相应的 push、pop、empty 函数

```
template<typename T>
struct MyStack {
    T* bottom;
    T* top_one;
    int max_len=10;
    MyStack()//构造函数
    {
        bottom = new T[max_len];
        top_one = bottom;
```

```

    }

    ~MyStack()//析构函数
        delete[] bottom;
};

```

2.1.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容
 # 不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明
 # 例如

```

MyStack::int push(T obj) //栈的 push 操作
{
    if (top_one - bottom == max_len) //栈满，则申请新栈，并把原内容复制后释
放原栈
    {
        max_len *= 2; //申请原栈两倍大的新栈
        T* mid = new T[max_len];
        memcpy(mid, bottom, sizeof(T)*(max_len/2));
        delete[] bottom; // 释放原栈
        bottom = mid;
        top_one = bottom + max_len;
    }
    *top_one++ = obj; //放入栈顶
    return OK;
}

MyStack::bool empty()//栈的判断是否为空的函数
{
    if (top_one == bottom) // 当栈顶栈底重合时，为空栈
        return true;
    else return false;
}

MyStack::T pop()//栈的删除栈顶函数
{
    if (top_one == bottom) // 空栈时报错
        throw runtime_error("Stack is empty!");
    return * (--top_one); // 不空时返回栈顶
}

```

```

    }
MyStack::T top()// 栈的取栈顶函数
{
    if (top_one == bottom) // 空栈时报错
        throw runtime_error("Stack is empty!");
    return *(top_one - 1); // 不空时删除并返回栈顶
}
int main()
{
    MyStack<char> input;
    char inp[100];
    cin.getline(inp, 100);
    for (int i = strlen(inp)-1; i>=0; i--)//输入入栈序列
    {
        input.push(inp[i]);
    }
    while (cin.getline(inp, 100))
    {
        MyStack<char> input_r=input;
        MyStack<char> answer;
        MyStack <char>station;
        for (int i = strlen(inp)-1; i >= 0; i--)//输入出栈序列
        {
            answer.push(inp[i]);
        }
        while (1)
        {
            // 入栈序列不空，且出栈栈顶和入栈栈顶相同
            if (!input_r.empty() && answer.top() == input_r.top())
            {
                answer.pop();
                input_r.pop();
            }
            // 车站栈不空，且车站栈顶和出栈栈顶相同
            else if (!station.empty() && station.top() == answer.top())
            {
                station.pop();
                answer.pop();
            }
            // 入栈序列不空，且入栈栈顶不同于出栈栈顶时，入栈栈顶进车站
            else if (!input_r.empty() && answer.top() != input_r.top())
            {

```

```

        station.push(input_r.top());
        input_r.pop();
    }
    // 此后 input_r 为空
    // 当车站非空，且车站栈顶等于出栈栈顶
    else if (input_r.empty() && !station.empty() && station.top() ==
answer.top())
    {
        station.pop();
        input_r.pop();
    }
    //station 非空且 answer 顶! =station 顶时，如果 inp 非空，什么都不做，
如果 inp 空，本方案不成立
    else if (input_r.empty() && !station.empty() && station.top() !=
answer.top())
    {
        cout << "no" << endl;
        break;
    }
    // 车站和入栈序列同时为空，说明全部出栈，本方案成立
    else if (input_r.empty() && station.empty())
    {
        cout << "yes" << endl;
        break;
    }
}
}
return 0;
}

```

2.1.5 调试分析（遇到的问题 and 解决方法）

简要描述调试过程

可以使用图片辅助说明

问题 1：尝试多个出栈序列时，如果用相同栈 input 存储入栈序列，只能用一次

解决 1：用 input_r 存储每次操作的入栈序列

问题 2：本地测试没有问题，oj 全部不通过

解决 2：排查输出，发现 No/Yes 大小写有问题

问题 3：pop、top 函数在空栈时的返回值不确定

解决 3：抛出异常

2.1.6 总结和体会

可以说说做此题的收获、分析这道题目的难点和易错点（数据边界、对算法效率的要求等）

本题收获：

1. 构造栈等数据结构时，如果涉及动态申请内存，要手动在析构函数中释放
2. 如果有返回值类型的函数中，不能确定返回值（出于某种非法情况），可以抛出异常

2.2 最长子串

2.2.1 问题描述

已知一个长度为 n ，仅含有字符 '(' 和 ')' 的字符串，请计算出最长的正确的括号子串的长度及起始位置，若存在多个，取第一个的起始位置。

子串是指任意长度的连续的字符序列。

例 1：对字符串 "(()())()" 来说，最长的子串是 "(()())"，所以长度=6，起始位置是 0。

例 2：对字符串 "()())" 来说，最长的子串是 "()", 子串长度=2，起始位置是 1。

例 3：对字符串 "" 来说，最长的子串是 ""，子串长度=0，空串的起始位置规定输出 0。

字符串长度： $0 \leq n \leq 1 \times 10^5$

对于 20% 的数据： $0 \leq n \leq 20$

对于 40% 的数据： $0 \leq n \leq 100$

对于 60% 的数据： $0 \leq n \leq 10000$

对于 100% 的数据： $0 \leq n \leq 100000$

2.2.2 基本要求

输入要求：一行字符串

输出要求：最长子串起始位置和长度

2.2.3 数据结构设计

```
template<typename T>
struct MyStack {
    T* bottom;
    T* top_one;
    int max_len = N;
public:
    MyStack()//构造函数
    {
        bottom = new T[max_len];
        top_one = bottom;
    }
    ~MyStack()//析构函数
    {
        delete[] bottom;
    }
};
```

2.2.4 功能说明（函数、类）

```
MyStack::int push(T obj) //栈的 push 操作
{
```

```
        if (top_one - bottom == max_len) //栈满，则申请新栈，并把原内容复制后释放原栈
```

```
    {
        max_len *= 2; //申请原栈两倍大的新栈
        T* mid = new T[max_len];
        memcpy(mid, bottom, sizeof(T)*(max_len/2));
        delete[] bottom; // 释放原栈
        bottom = mid;
        top_one = bottom + max_len;
    }
    *top_one++ = obj; //放入栈顶
    return OK;
}
```

```
MyStack::bool empty()//栈的判断是否为空的函数
```

```
{
    if (top_one == bottom) // 当栈顶栈底重合时，为空栈
        return true;
    else return false;
}
```

```
MyStack::T pop()//栈的删除栈顶函数
```

```
{
    if (top_one == bottom) // 空栈时报错
        throw runtime_error("Stack is empty!");
    return * (--top_one); // 不空时返回栈顶
}
```

```
MyStack::T top()// 栈的取栈顶函数
```

```
{
    if (top_one == bottom) // 空栈时报错
        throw runtime_error("Stack is empty!");
    return *(top_one - 1); // 不空时删除并返回栈顶
}
```

```
char inp[N];
```

```
int main()
```

```
{
```

```

cin.getline(inp, N);    //输入字符串
int len = strlen(inp);
int max_len = -1;
char* max_len_pos = NULL;
char* cur_ptr = inp;    //取指针初始值
while (1)
{
    if (cur_ptr >= inp + len)// 指针越界时退出
        break;
    stack<char> test;
    // 忽略字符直到下一个 '('
    while (*cur_ptr != '(' && *cur_ptr != '\0')
        cur_ptr++;
    if (*cur_ptr == '\0') // 指针越界时退出
        break;
    char* start_ptr = cur_ptr;
    test.push(*cur_ptr++);
    int cur_max_len = -1;
    int cur_len = 1;

    int ok = 1;
    //从 cur_ptr 到串尾，寻找最长连续字符串
    while (*cur_ptr != '\0')
    {
        // 当栈顶和当前指针匹配，pop 栈顶，长度加一
        if (*cur_ptr == ')' && !test.empty() && test.top() == '(')
        {
            test.pop();
            cur_len++;
        }
        // 栈顶和当前指针不匹配，当前指针项入栈，长度++
        else
        {
            test.push(*cur_ptr);
            cur_len++;
        }
        // 指针指向下一位
        cur_ptr++;
        // 当栈空，表示匹配到一个连续字符串，记录当前字符串长度
        if (test.empty())
            cur_max_len = cur_len;
    }
    // 更新最长字符串长度、最长字符串起始位置
    if (max_len < cur_max_len)

```

```

    {
        max_len = cur_max_len;
        max_len_pos = start_ptr;
    }
    // 更新 cur_ptr
    if (cur_max_len > 0) // 若匹配到连续字符串，跳过这个字符串内部进行匹配（因为从字符串内部开始的连续字符串长度不可能超过原本的连续字符串）
        cur_ptr = start_ptr + cur_max_len;
    else // 若没有匹配到连续字符串，则从下一位字符开始匹配
        cur_ptr = start_ptr + 1;
}

if (max_len == -1)
{
    max_len = 0;
    max_len_pos = inp;
}

// 输出结果
printf("%d %d\n", max_len, max_len_pos - inp);
return 0;
}

```

2.2.5 调试分析（遇到的问题 and 解决方法）

问题 1：问题解决逻辑不清晰

解决 1：先列出大量 if-else 从句，随后逐步缩短精简

2.2.6 总结和体会

应当有更好的算法：对于每一对匹配的括号，计算他们之间的距离即可 ($O(n)$ ，本方法则是 $O(n^2)$)

2.3 布尔表达式

2.3.1 问题描述

计算如下布尔表达式 $(V|V) \& F \& (F|V)$ 其中 V 表示 True，F 表示 False，| 表示 or，& 表示 and，! 表示 not（运算符优先级 not > and > or）

2.3.2 基本要求

输入要求：

文件输入，有若干 ($A \leq 20$) 个表达式，其中每一行为一个表达式。表达式有 ($N \leq 100$) 个符号，符号间可以用任意空格分开，或者没有空格，所以表达式的总长度，即字符的个数，是未知的。

对于 20% 的数据，有 $A \leq 5$ ， $N \leq 20$ ，且表达式中包含 V、F、&、|

对于 40% 的数据，有 $A \leq 10$ ， $N \leq 50$ ，且表达式中包含 V、F、&、|、!

对于 100%的数据，有 $A \leq 20$ ， $N \leq 100$ ，且表达式中包含 V、F、&、|、!、(、)

所有测试数据中都可能穿插空格

输出要求：

对测试用例中的每个表达式输出“Expression”，后面跟着序列号和“:”，然后是相应的测试表达式的结果（V 或 F），每个表达式结果占一行（注意冒号后面有空格）。

2.3.3 数据结构设计

```
struct MyStack {
    T* bottom;
    T* top_one;
    int max_len = N;

public:
    MyStack() //构造函数
    {
        bottom = new T[max_len];
        top_one = bottom;
    }
    ~MyStack() //析构函数
        delete[] bottom
};
```

2.3.4 功能说明（函数、类）

```
//push、pop、top 操作同上，在此不重复列举

MyStack::int size() //返回栈大小
    return top_one - bottom;
MyStack::void clear()//清空栈
    top_one = bottom;

void Switch(char& c1)// 切换 c1 的真值
{
    if (c1 == 'V')
        c1 = 'F';
    else if (c1 == 'F')
        c1 = 'V';
    else return;
}

//用 operator1 运算栈 values
void Operate_by_operator(char operator1, MyStack<char>*
values, MyStack<char>* operators)
{
    char op1, op2;
```

```

// 右括号的运算：取运算符进行运算直到遇见左括号
// 左括号的运算：无
if (operator1 == ')')
{
    if (operators->top() != '(')
    {
        Operate_by_operator(operators->pop(), values, operators);
        operators->push(')');
    }
    else
        operators->pop();
//NOT 运算：取 values 栈顶，取反后放回
else if (operator1 == NOT)
{
    op1 = values->pop();
    Switch(op1);
    values->push(op1);
}
//AND 运算：取 values 栈顶两值作 and 逻辑运算
else if (operator1 == AND)
{
    op1 = values->pop();
    op2 = values->pop();
    values->push((op1 == 'V' && op2 == 'V')?'V':'F');
}
//OR 运算：取 values 栈顶两值作 or 逻辑运算
else if (operator1 == OR)
{
    op1 = values->pop();
    op2 = values->pop();
    values->push((op1 == 'V' || op2 == 'V') ? 'V' : 'F');
}
}

//返回前者优先级大于后者的 bool 值,后者在前者的右侧
bool cmp_priority(char c1, char c2)
{
    switch (c1)
    {
        case('('):
            c1 = -1;
            break;
        case(')'):
            c1 = 10;
    }
}

```

```

        break;
    case('!'):
        c1 = 3;
        break;
    case('&'):
        c1 = 2;
        break;
    case('|'):
        c1 = 1;
        break;
    default:
        break;
}

switch (c2)
{
    case('('):
        c2 = 10;
        break;
    case(')'):
        c2 = -1;
        break;
    case('!'):
        c2 = 3;
        break;
    case('&'):
        c2 = 2;
        break;
    case('|'):
        c2 = 1;
        break;
    default:
        break;
}
return c1 > c2;
}

int main()
{
    int kase = 1; //表达式计数
    char cmd[MAX_LEN];
    while (cin.getline(cmd, MAX_LEN))
    {
        //入栈

```

```

    for (int i = 0; i < strlen(cmd); i++)
    {
        if (cmd[i] == ' ')
            continue;
        //值入栈
        else if (cmd[i] == 'V' || cmd[i] == 'F')
            values.push(cmd[i]);
        // 运算符运算或入栈
        else{
            while
(!operators.empty()&&cmp_priority(operators.top(),cmd[i] ))//要入站的优先级
低

            Operate_by_operator(operators.pop(),&values,&operators);
            operators.push(cmd[i]);
        }
    }
    // 表达式完全入栈后，逐个求值
    while (!operators.empty())
    {
        Operate_by_operator(operators.pop(),&values, &operators);
    }
    // 打印结果
    printf("Expression %d: %c\n", kase++, values.pop());
}
return 0;
}

```

2.3.5 调试分析（遇到的问题解决方法）

问题 1：括号的运算法不明确

解决 1：分类讨论

问题 2：优先级不能直接表示

解决 2：设计 cmp 函数，实现两个运算符的比较，同时注意运算符左右顺序

2.3.6 总结和体会

体会 1：可以利用两个栈：运算符栈和值栈，实现有优先级的表达式求值。

体会 2：在实现过程中，需要编写大量的测试用例来验证代码的正确性，特别是在处理复杂表达式和边界条件时，需要仔细调试和测试。

体会 3：要注意边界条件，在处理括号和运算符时，需要注意边界条件，如空栈和运算符不匹配的情况，仔细处理这些边界条件可以避免运行时错误。

2.4 队列的应用：寻找特定连通块

2.4.1 问题描述

输入一个 $n \times m$ 的 0 1 矩阵，1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，仅在矩阵边缘联通的不算作区域。求区域数。此算法在细胞计数上会经常用到。对于所有数据， $0 \leq n, m \leq 1000$

2.4.2 基本要求

输入要求：

第 1 行 2 个正整数 n, m ，表示要输入的矩阵行数和列数

第 2— $n+1$ 行为 $n \times m$ 的矩阵，每个元素的值为 0 或 1。

输出要求：

1 行，代表区域数

2.4.3 数据结构设计

```
template<typename T>
struct My_Queue {           //一个链表实现的线性队列
    struct Node {
        T elem;
        Node* next;
    };
    Node* _front;
    Node* _back;
    My_Queue() //构造函数
    {
        _front = new Node; //头结点
        _back = _front;
        _back->next = NULL;
    }
    ~My_Queue() //析构函数
    {
        while (_back != _front)
        {
            Node* mid = _front->next;
            delete _front;
            _front = mid;
        }
    }
};
```

2.4.4 功能说明（函数、类）

```
MyQueue::bool empty() //队列的检查是否为空函数
{
    if (_front == _back)
        return true;
```

```

        return false;
    }
MyQueue::T front() //队列的观察首项的函
    return _front->next->elem;
MyQueue::void pop()    //队列的 pop 操作
{
    if (_front != _back)
    {
        Node* new_front = _front->next->next;
        if (_front->next == _back)
            _back = _front;
        delete _front->next;
        _front->next = new_front;
    }
}
MyQueue::void push(T new_elem)    //队列的 push 操作
{
    Node* new_node = new Node;
    new_node->elem = new_elem;
    new_node->next = NULL;
    _back->next = new_node;
    _back = new_node;
}
//相邻的 1 置零，返回连通块数量
int bfs_for_connected_1(My_Queue<int> &q1, int(&matrix)[N][N])
{
    int cnt = 0;
    while (!q1.empty())
    {
        int pos_num = q1.front();
        q1.pop();
        //越界处理
        if (pos_num < 0 || pos_num >= m * n)
            continue;
        cnt++;
        int col = pos_num % n; //根据序号求行列
        int row = pos_num / n;
        matrix[row][col] = 0;
        if (row + 1 >= 0 && row + 1 < m && matrix[row + 1][col] ==
1)q1.push(pos_num + n);
        if (row - 1 >= 0 && row - 1 < m && matrix[row - 1][col] ==
1)q1.push(pos_num - n);
        if (col + 1 >= 0 && col + 1 < n && matrix[row][col + 1] ==
1)q1.push(pos_num + 1);
    }
}

```

```

        if (col - 1 >= 0 && col - 1 < n && matrix[row][col - 1] ==
1)q1.push(pos_num - 1);
    }
    return cnt;
}
int main()
{
    scanf("%d%d", &m, &n); //m 行 n 列
    My_Queue<int> q1;
    int cnt_all = 0;
    for(int i=0;i<m;i++)
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &matrix[i][j]);
            matrix2[i][j] = matrix[i][j];
        }
    //计数所有连通块
    for (int i = 0; i < m * n; i++)
    {
        if (matrix[i / n][i % n] == 0)
            continue;
        else
        {
            q1.push(i);
            bfs_for_connected_1(q1,matrix);
            cnt_all++;
        }
    }
    //减去完全在边缘中的连通快数量
    int ptr = 0;
    //构造顺时针边缘序号集
    for (int i = 0; i < n-1; ptr++, i++)
        seq[ptr] = i;
    for (int i = n - 1; i < n * m-1; ptr++, i += n)
        seq[ptr] = i;
    for (int i = n * m - 1; i > n * (m - 1); ptr++, i--)
        seq[ptr] = i;
    for (int i = n * (m - 1); i > 0; ptr++, i-=n)
        seq[ptr] = i;
    //先检查 0,0 处是否有连通块
    if (matrix2[0][0] == 1)
    {
        q1.push(0);
        int cnt_for_0 = 0;

```

```

        for (int i = 0; matrix2[0][i] == 1; i++, cnt_for_0++);
        for (int j = 1; matrix2[j][0] == 1; j++, cnt_for_0++);
        int num_0 = bfs_for_connected_1(q1, matrix2);
        if (num_0 == cnt_for_0)
            cnt_all--;
    }
    //顺时针检查完全在边缘中的连通块数量
    for (int i = 0; i < ptr; i++)
    {
        if (matrix2[seq[i]/n][seq[i] % n] == 0)
            continue;
        else//等于1
        {
            int start = i;
            while (i <= ptr && matrix2[seq[i] / n][seq[i] % n] == 1)i++;
            int length = i - start;
            q1.push(seq[i-1]);
            int connects=bfs_for_connected_1(q1, matrix2);
            if (connects == length)
                cnt_all--;
        }
    }
    printf("%d", cnt_all);
    return 0;
}

```

2.4.5 调试分析（遇到的问题和解决方法）

问题 1：难以直接计数“不完全在边缘连通的连通块”，理解也有困难

解决 1：尝试，同时把上式表示为 所有连通块-仅在边缘中的连通块

问题 2：考虑仅在边缘中的连通块时，起点位置的逆时针连通块没有考虑

解决 2：单独考虑从起点位置开始的边缘连通块

2.4.6 总结和体会

体会 1：分类讨论是解决问题的好办法，可以先解决问题的大部分，再一一讨论剩下的边界条件，独特处理

体会 2：直接求答案困难时，可以把答案表示成多个易求值的运算结果，分别求取后运算。

2.5 队列中的最大值

2.5.1 问题描述

给定一个队列，有下列 3 个基本操作：

- (1) Enqueue(v): v 入队
- (2) Dequeue(): 使队首元素删除，并返回此元素
- (3) GetMax(): 返回队列中的最大元素

请设计一种数据结构和算法，让 GetMax 操作的时间复杂度尽可能地低。

2.5.2 基本要求

输入要求：

第 1 行 1 个正整数 n，表示队列的容量(队列中最多有 n 个元素)，接着读入多行，每一行执行一个动作。

若输入"dequeue"，表示出队，当队空时，输出一行"Queue is Empty";否则，输出出队的元素；

若输入"enqueue m"，表示将元素 m 入队,当队满时(入队前队列中元素已有 n 个)，输出"Queue is Full"，否则，不输出；

若输入"max",输出队列中最大元素，若队空，输出一行"Queue is Empty"。

若输入"quit",结束输入，输出队列中的所有元素

输出要求：

多行，分别是执行每次操作后的结果

2.5.3 数据结构设计

```
template<typename T>
struct My_Queue {          //一个顺序表实现的循环队列
    T* Mem_Beginer;        //分配的内存的起点
    T* Mem_Ending;         //分配的内存的终点
    int Mem_MaxLen;        //分配的内存的长度
    T* _front;             //队列起点
    T* _back;              //队列终点
    //使用头结点表示空队列
    My_Queue(int iMaxLen)   //构造函数
    {
        Mem_Beginer = new T[iMaxLen+1];
        Mem_Ending = Mem_Beginer + iMaxLen;
        Mem_MaxLen = iMaxLen;
        _front = Mem_Beginer;
        _back = Mem_Beginer;
    }
    ~My_Queue()             //析构函数
    {
        delete Mem_Beginer;
    }
};
```

2.5.4 功能说明（函数、类）

//pop、push、empty 函数同上，这里略去不写

//返回队列中元素在队列中的下一个元素

```
MyQueue::T* next(T* ptr)
{
    if (ptr == Mem_Ending)
        return Mem_Beginer;
    else
```

```

        return ptr + 1;
    }
    //顺序遍历队列，返回队列中的最大值
    MyQueue::void getMax()
    {
        if (empty())
            printf("Queue is Empty\n");
        else
        {
            T* cur_ptr = next(_front);
            T* End_next = next(_back);
            int max = MIN;
            while (cur_ptr != End_next)
            {
                if (*cur_ptr > max)
                    max = *cur_ptr;

                cur_ptr = next(cur_ptr);
            }
            printf("%d\n", max);
        }
    }
    //结束处理
    MyQueue::void Quit()
    {
        T* cur_ptr = next(_front);
        T* End_next = next(_back);
        int max = MIN;
        while (cur_ptr != End_next)
        {
            printf("%d ", *cur_ptr);
            cur_ptr = next(cur_ptr);
        }
        printf("\n");
    }
    int main()
    {
        int n;
        scanf("%d", &n);
        My_Queue<int> q1(n);
        char cmd[100];

        int ok = 1;
        while (cin.getline(cmd,100))//读取命令，根据首字母处理

```

```

{
    switch (cmd[0])
    {
        case('q'):
            ok = 0;
            q1.Quit();
            break;
        case('m'):
            q1.getMax();
            break;
        case('e'):
            int n;
            char cmd2[100];
            sscanf(cmd, "%s%d", &cmd2, &n);
            q1.push(n);
            break;
        case('d'):
            q1.pop();
            break;
        default:
            break;
    }
    if (!ok)
        break;
}
return 0;
}

```

2.5.5 调试分析（遇到的问题 and 解决方法）

问题 1: front 和 back 可能出现后者小于前者的情况，不能使用 `i++` 遍历队列

解决 1: 使用 next 成员函数遍历队列

问题 2: 希望用头结点存储最大值，但是当 pop 后，不能得知剩下队列中的最大值

解决 2: 用顺序表，通过 $O(n)$ 的遍历，找到最大值

2.5.6 总结和体会

体会 1: 可以用整型值代替指针表示位置，从而用计算代替调用成员函数 next

体会 2: 顺序表对于随机存取的效率高于链表

3. 实验总结

应用场景: 栈适合需要回溯的场景（如递归、表达式求值），队列适合需要按顺序处理的场景（如任务调度、BFS）。

实现方式: 栈通常使用数组或链表实现，队列也可以用数组（环形缓冲区）或链表实现。

注意事项: 要注意构造和析构函数的对应，有动态内存申请的，要注意释放空间